Custom Integrations

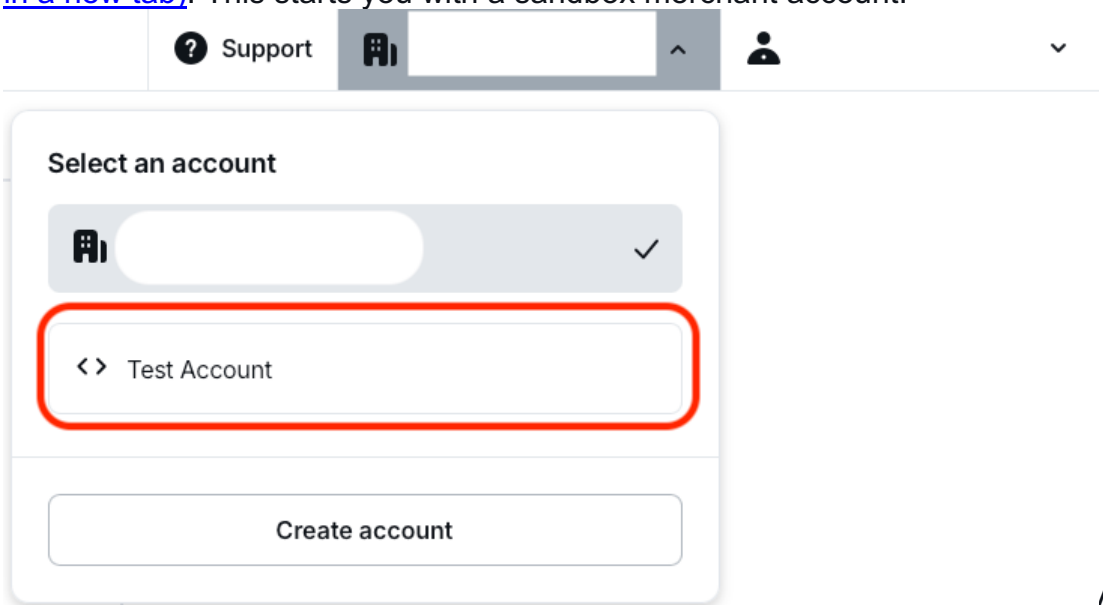
Integration guides for building with SumUp SDKs and APIs.
Guides in this section cover the basics of SumUp APIs, allowing you to develop a custom integration if SumUp products don't fully support your use case.

Getting a Sandbox Merchant Account

Create a sandbox merchant account to test APIs without real money:

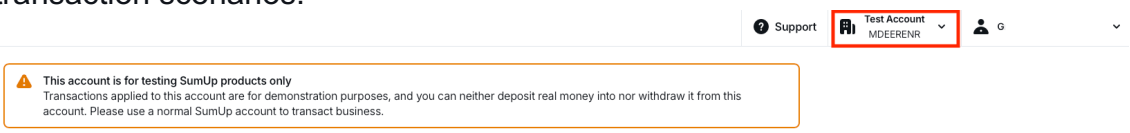
1. Log in to your SumUp account.
2. Open Developer Settings(Opens in a new tab).
3. In the **Sandboxes** tab, create a sandbox merchant account if you do not have one yet.

Note
If you don't have a sandbox account yet, sign up for a developer account(Opens in a new tab). This starts you with a sandbox merchant account.



A screenshot of the account selection dropdown in the SumUp dashboard with the sandbox merchant account highlighted with red circle

With your sandbox merchant account, begin making API calls with real data. Sandbox merchant accounts **do not** process transactions with real funds. The sandbox merchant account has a different ID and displays a clear warning. Requests with a value of 11 (in any currency) always fail by design, to test failed transaction scenarios.



A screenshot of the dashboard with sandbox merchant account selected

When finished experimenting with the sandbox merchant account, switch back to a regular account for business purposes.

SumUp provides official SDKs for Node.js, Go, Python, Java, PHP, .NET, and Rust — visit the SDKs overview page to choose the client that fits your stack.

Accept a Payment

Create and process a checkout for a single card payment.

Overview

This guide covers single payments over SumUp API, using customer-entered card details without associating it with a specific customer saved for the merchant user's account. Ideal for quick checkout processing if you don't want to use our Payment Widget.

You will go through the following steps:

1. Create a checkout
2. Complete a checkout

When you complete these steps, you will have processed a payment with a payment card entered by a customer.

Prerequisites

- SumUp merchant account(Opens in a new tab) with completed account details(Opens in a new tab).
    - You can also use a sandbox merchant account.
- Registered client application with SumUp.
- Valid access token obtained either with the Authorization code flow or Client credentials flow.

Steps

1. Create a Checkout

To create a new checkout resource, make a POST request to the `https://api.sumup.com/v0.1/checkouts` endpoint.

Caution

Checkout request must be made in a server-to-server communication between your server and SumUp API.

Note

The table below lists only the **required** parameters for the request. For a full list, see the API Reference.

Attribute name	Description
checkout_reference	A unique string identifying the payment in your system. It must be unique for your application.
amount	The amount for which the payment is made.

Attribute name	Description
currency	The three-letter ISO4217(Opens in a new tab) code of the payment currency. It must match the currency registered for the specified merchant (see merchant_code).
merchant_code	The code of the registered SumUp merchant the payment is made.

Example request:

- **cURL**
- Node.js
- .NET
- Java
- Go
- Python
- Rust
- PHP

```
curl -X POST \
  https://api.sumup.com/v0.1/checkouts \
  -H "Authorization: Bearer $SUMUP_API_KEY" \
  -H 'Content-Type: application/json' \
  -d '{
      "merchant_code": "ME7RMQN3",
      "amount": 15.0,
      "currency": "EUR",
      "checkout_reference": "unique-checkout-ref-123"
    }'
```

The response contains a JSON body with the details of the created checkout resource. Note the identifier of the resource - you will need it for all operations on the resource, such as processing the payment in the next step.

The resource identifier is returned in the `id` attribute, or `f14425c6-8bc1-4d02-957c-47573f762828` in the sample response below. The `status` attribute shows that the payment for this resource is pending and will be processed when you complete the checkout with the payment instrument details.

```
{
  "checkout_reference": "CO746453",
  "amount": 10,
  "currency": "EUR",
  "merchant_code": "ME7RMQN3",
  "description": "Sample one-time payment",
  "id": "f14425c6-8bc1-4d02-957c-47573f762828",
  "status": "PENDING",
  "date": "2018-09-28T13:47:01.832Z",
  "transactions": []
}
```

2. Complete a Checkout

To process the checkout and trigger the payment, make a PUT request to the `https://api.sumup.com/v0.1/checkouts/{id}` endpoint, where the value of the `{id}` path parameter is the identifier of the checkout resource.

Caution

This request must be made from your client, *not* from your server.

The request body should be a JSON object with the payment type (card) and the payment card details. For more information about the request, see the API Reference.

Example request for Step 1 checkout:

- **cURL**
- Node.js
- .NET
- Java
- Go
- Python
- Rust
- PHP

```
curl -X PUT \
 https://api.sumup.com/v0.1/checkouts/f14425c6-8bc1-4d02-957c-47573f762828 \
 -H "Authorization: Bearer $SUMUP_API_KEY" \
 -H 'Content-Type: application/json' \
 -d '{
     "payment_type": "card",
     "card": {
      "name": "Boaty McBoatface",
      "number": "4200000000000042",
      "expiry_month": "12",
      "expiry_year": "23",
      "cvv": "123"
     }
    }'
```

The response contains a JSON body with the details of the processed checkout resource in which the attributes related to the payment outcome are populated:


```
{
 "checkout_reference": "CO746453",
 "amount": 10.0,
 "currency": "EUR",
 "merchant_code": "ME7RMQN3",
 "description": "Sample one-time payment",
 "id": "f14425c6-8bc1-4d02-957c-47573f762828",
 "status": "PAID",
 "date": "2018-09-28T13:47:01.832Z",
 "transaction_code": "TFDCP3FLQ7",
 "transaction_id": "19aa3cca-89f6-42d2-b462-463b0b53e959",
 "transactions": [
  {
   "id": "19aa3cca-89f6-42d2-b462-463b0b53e959",
   "transaction_code": "TFDCP3FLQ7",
   "merchant_code": "ME7RMQN3",
   "amount": 10.0,
   "vat_amount": 0.0,
   "tip_amount": 0.0,
   "currency": "EUR",
   "timestamp": "2018-09-28T13:48:28.768Z",
   "status": "SUCCESSFUL",
   "payment_type": "ECOM",
   "entry_mode": "CUSTOMER_ENTRY",
   "installments_count": 1,
   "auth_code": "585388",
```

"internal_id": 24118507
}
]
}

Result

You have made a payment with a payment card entered by a customer without associating it with a specific customer saved for a merchant user's account. If the payment is successful, the funds will be transferred to the merchant user according to the configured account settings.

Refunds

Walks through looking up transactions and issuing full or partial refunds.

Overview

In this guide, you will learn how to refund a transaction. You will go through the following steps:

1. Look up a transaction ID (Optional)
2. Refund a transaction by using one of the available options:
    - Option A: Make a full refund
    - Option B: Make a partial refund

When you complete these steps, the payment you have previously processed through SumUp will be refunded either partially or in full.

Before You Begin

Here are the things that you need in order to complete the steps in this guide:

- SumUp merchant account(Opens in a new tab) with completed account details(Opens in a new tab).
    - You can also use a sandbox merchant account.
- Registered client application with SumUp.
- Valid access token obtained with the Authorization code flow.
- You have processed a checkout and you have the checkout ID.

Note
Transactions are associated with active merchant user accounts. As a result, you *cannot* use an access token obtained via the Client credentials flow to complete the steps in this guide.

Steps

1. Look up a Transaction ID

Note
If you already have the ID of the transaction you want to refund, you can skip this step and continue with Step 2.

1. Make a GET request to the `https://api.sumup.com/v0.1/checkouts/{id}` endpoint, where the value of the `{id}` path parameter is the identifier of the checkout resource.

Example request:

- **cURL**
- Node.js
- .NET
- Java
- Go
- Python
- Rust
- PHP

```
curl -X GET \
 https://api.sumup.com/v0.1/checkouts/4ebc2ed7-bb8c-4d4d-a110-08fd31301bf2 \
 -H "Authorization: Bearer $SUMUP_API_KEY"
```

The response contains a JSON body with the full details of the processed checkout resource. You can find the transaction ID in the `id` attribute of the respective transaction resource (`664200af-2b62-4142-9c73-a2a505310d78` in the sample response below).

```
{
 "checkout_reference": "CO287866",
 ...
 "id": "4ebc2ed7-bb8c-4d4d-a110-08fd31301bf2",
 ...
 "transactions": [
  {
   "id": "664200af-2b62-4142-9c73-a2a505310d78",
   ...
  }
 ]
}
```

2. Refund a Transaction

- Option A: Make a full refund
- Option B: Make a partial refund

Make a Full Refund

1. Make a POST request with an empty request body to the `https://api.sumup.com/v0.1/me/refund/{txn_id}` endpoint, where the value of the `{txn_id}`path parameter is the identifier of the transaction resource.

Example request for the transaction with identifier `664200af-2b62-4142-9c73-a2a505310d78`:

- **cURL**
- Node.js
- .NET
- Java
- Go
- Python
- Rust

- PHP

```
curl -X POST \
  https://api.sumup.com/v0.1/me/refund/19aa3cca-89f6-42d2-b462-463b0b53e959 \
  -H "Authorization: Bearer $SUMUP_API_KEY"
```

The response returns a 204 HTTP status code and contains no body.

Make a Partial Refund

1. Make a POST request to the `https://api.sumup.com/v0.1/me/refund/{txn_id}` endpoint, where the value of the `{txn_id}` path parameter is the identifier of the transaction resource.

Unlike the option for a full refund, the request body for partial refunds should be a JSON object with the amount you want to refund for the transaction.

Example request for a partial refund for the amount of 24.42 EUR:

- **cURL**
- Node.js
- .NET
- Java
- Go
- Python
- Rust
- PHP

```
curl -X POST \
  https://api.sumup.com/v0.1/me/refund/19aa3cca-89f6-42d2-b462-463b0b53e959 \
  -H "Authorization: Bearer $SUMUP_API_KEY" \
  -H 'Content-Type: application/json' \
  -d '{"amount": 24.42}'
```

The response returns a 204 HTTP status code and contains no body.

Result

You have successfully refunded a transaction (either partially or in full) for a payment you previously processed. The refunded amount will be credited to the same payment method the customer had used to pay with in the original transaction. The processing fees associated with the original transaction are not returned.

Response Handling

Explains how to interpret SumUp API responses.

Overview

Once an API call is submitted to SumUp, you will receive a standard HTTP response informing you if your request has been successful or not. In most

cases the HTTP codes are accompanied by a response body in JSON(Opens in a new tab) format.

Codes in the 2xx range indicate success,4xx range indicates errors where the information provided results in a failure. Codes in the 5xx range are rare and indicate server-side errors.

Successful Requests

Successfully processed requests return one of the following HTTP status codes: 200 OK, 201 Created, 202 Accepted, 204 No Content. Depending on the use case response bodies may not be present at all. A response body can look like this:


```
{
 "next_step": {
  "url": "https://dummy-3ds-gateway.com/cap?RID=1233&VAA=A",
  "method": "POST",
  "redirect_url": "https://mysite.com/completed_purchase",
  "mechanism": "iframe",
  "payload": {
   "PaReq":
"eJxVUttu2zAM/RXDr4MjyY5dO6BVuE27FZuDZHGG9VGRmMSFb/..f16+jLt/gPhUvGGw==",
   "MD": "b1a536c0-29b9-11eb-adc1-0242ac120002"
  }
 }
}
```


Client-Side Issues

4xx are occasions where you can take action to correct your application requests. Typically, these codes are returned upon user error, such as invalid API calls, incorrect values, missing parameters, etc. The response you receive will provide an indication to the root cause that triggered it. Here's an example of a 4xx response:


```
{
 "message": "Validation error",
 "error_code": "MISSING",
 "param": "merchant_code"
}
```


Server-Side Issues

5xx stand for server errors. Although rare, if you receive such code, we recommend retrying your request. Should the returned codes continue to be in the 5xx range, reach out to us.

Save Customer Cards

Tokenize cards with the Payment Widget for future recurring payments.

About Card Tokenization

In this guide, you will learn how to use the Payment Widget to store card details for future use, and how to set up recurring payments, as SumUp handles the UI/UX and consent collection for you. Alternatively, you can process payments with your own UI, following the API guidelines also provided in this document. This feature is also known as **card on file** or **tokenization**.

You will go through the following steps:

1. Create a customer.
2. Create checkout intended for card tokenization, where we specify it's for tokenization purpose. This is where 3DS verification takes place. The transaction amount is instantly reimbursed.
3. Process payment with the payment widget or process payment directly with SumUp API.
4. Retrieve the tokenized card.
5. Make subsequent payments with the tokenized card.

Prerequisites

- You have a merchant account with SumUp(Opens in a new tab) and have already filled in your account details(Opens in a new tab).
    - You can also create a sandbox merchant account. Please note that setting up 3DS verification in a sandbox merchant account requires contact with our team at integrations@sumup.com.
- You have an API Key. For more details see the Authorization Guide.
- You have control over the backend server to retrieve data securely.

Creating Customer

A customer resource is a representation of a person or business paying for a product or service. It contains personal information such as name, contact details, postal address, as well as a unique identifier relevant to your business logic (`customer_id`).

1. Create a new customer resource with a POST request to the `https://api.sumup.com/v0.1/customers` endpoint:

- **cURL**
- Node.js
- .NET
- Java
- Go
- Python
- Rust
- PHP

```
curl -X POST \
  https://api.sumup.com/v0.1/customers \
  -H "Authorization: Bearer $SUMUP_API_KEY" \
  -H 'Content-Type: application/json' \
  -d '{
  "customer_id": "MYCUSTOMERID-123",
  "personal_details": {
    "address": {
      "city": "Venice",
      "state": "California",
```

```
    "country": "US",
    "line1": "606 Venezia Ave",
    "line2": "Front",
    "postal_code": "90291"
  },
  "birthdate": "1949-11-11",
  "email": "thedude@example.com",
  "first_name": "Jeffrey",
  "last_name": "Lebowski",
  "phone": "+1 310-555-1234"
 }
}'
```

You should expect a standard `201 Created` response, with the customer details you passed. For full details, see the [endpoint documentation]. Having created the customer, we can now proceed to making a payment

Creating Checkout to Save Card

Now, we need to tokenize the customer's card, and we will need a checkout for this. The checkout resource is a representation of a payment being made by the previously created customer. It contains information such as the amount, currency, and a unique `checkout_reference` identifier that is relevant to your business logic.

The flow is initiated with the `create a checkout` endpoint. It is important to pass the `customer_id`parameter in this step, for future linking to a payment instrument. Critically, a `purpose` parameter is passed to indicate the payment type as **recurring payment** and process an authorization charge of the checkout amount indicated, **which is instantly reimbursed**. Note that this doesn't automatically imply further payments from this customer - at this point, we're just tokenizing the card.

1. To create a new checkout resource, make a POST request to the `https://api.sumup.com/v0.1/checkouts` endpoint.

Example of such request:

- **cURL**
- Node.js
- .NET
- Java
- Go
- Python
- Rust
- PHP

```
curl -X POST \
 https://api.sumup.com/v0.1/checkouts \
 -H "Authorization: Bearer $SUMUP_API_KEY" \
 -H 'Content-Type: application/json' \
 -d '{
     "checkout_reference": "MYCHECKOUT",
     "amount": 1,
     "currency": "EUR",
     "merchant_code": "MDEERENR",
     "description": "My checkout",
```

```
    "customer_id": "MYCUSTOMERID-123",
    "purpose": "SETUP_RECURRING_PAYMENT"
  }'
```

You should expect a standard `201 Created` response, with the checkout reference and both merchant and customer information.


```
{
  "amount": 1,
  "checkout_reference": "MYCHECKOUT",
  "checkout_type": "checkout",
  "currency": "EUR",
  "customer_id": "MYCUSTOMERID-123",
  "date": "2025-10-29T15:09:11.550+00:00",
  "description": "My checkout",
  "id": "7164c99b-13cb-42a1-8ba1-3c2c46a29de7",
  "merchant_code": "MDEERENR",
  "merchant_country": "PL",
  "merchant_name": "Sandbox Merchant Account",
  "pay_to_email": "a8e019f9bb2f49159182e8bd61eb5ea6@developer.sumup.com",
  "purpose": "SETUP_RECURRING_PAYMENT",
  "status": "PENDING",
  "transactions": []
}
```


For more information, see the [create a checkout]() endpoint.

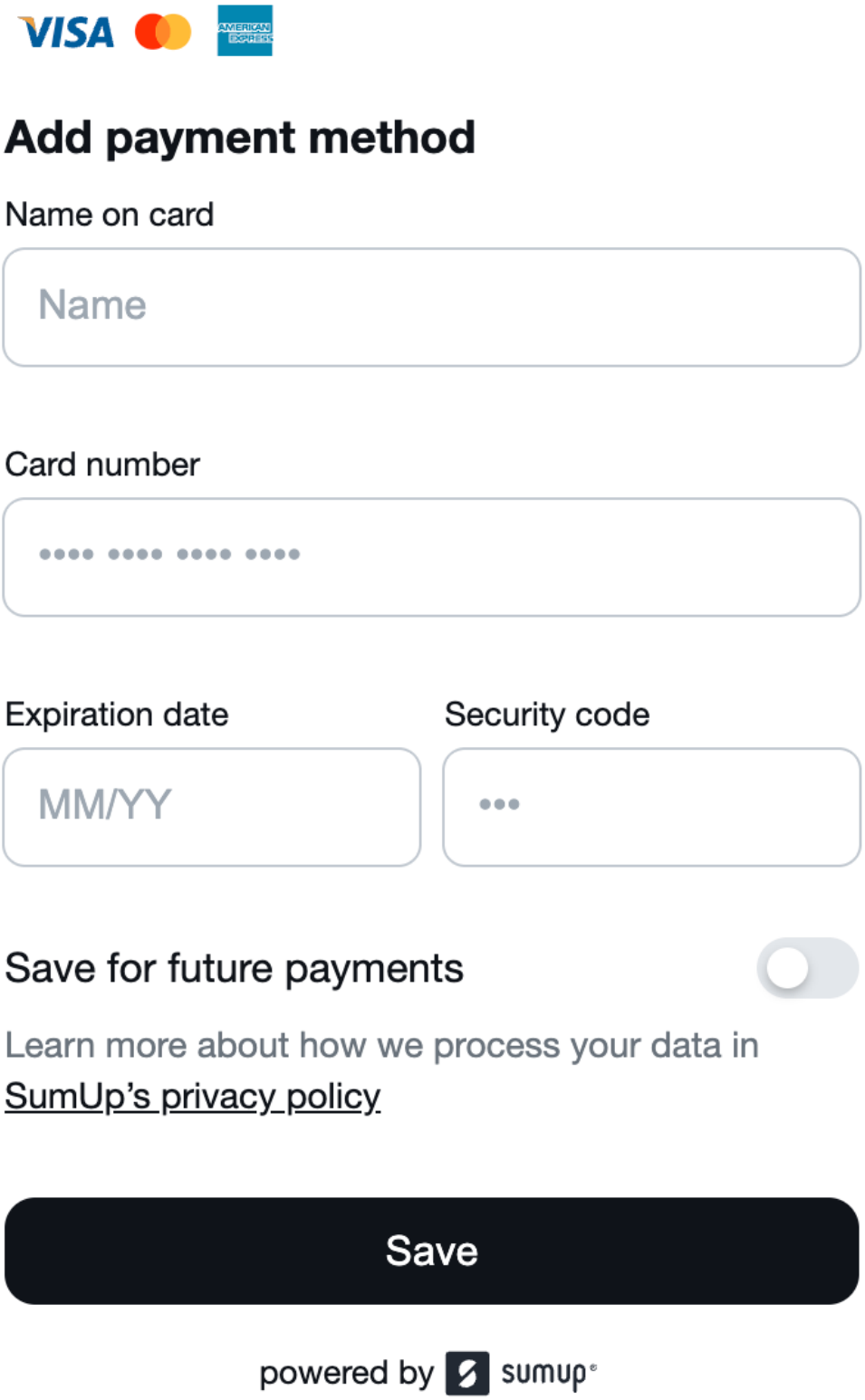
Processing Request with Payment Widget

The [SumUp's Payment Widget]() is a JavaScript library that allows you to securely process checkouts and collect card details securely, handling consent collection and 3DS verification. These steps are mandatory due to legal requirements, which is why we recommend processing the checkout with our widget.

Once you have a checkout reference from the step above, you can mount the payment widget into your website and collect the card details. Pass the `checkout_reference` from above as the `checkoutId` here.

```
<div id="sumup-card"></div>
<script
 type="text/javascript"
 src="https://gateway.sumup.com/gateway/ecom/card/v2/sdk.js"
></script>
<script type="text/javascript">
 SumUpCard.mount({
  id: "sumup-card",
  checkoutId: `${checkout_id}`, // Ex: '7164c99b-13cb-42a1-8ba1-3c2c46a29de7'
  onResponse: function (type, body) {
    console.log("Type", type);
    console.log("Body", body);
  },
 });
</script>
```

Upon mounting the Payment Widget with a recurring purpose checkout, you should see the following screen:



Card on file with payment widget

The user is prompted to enter their card details, give consent to store their card details and complete the checkout. The widget uses the [Process Checkout endpoint] to securely process the checkout. We strongly recommend using the

widget over direct integration, as the widget correctly handles all legal requirements for payments.

If the previous operation is successful, and the card is stored with the **Save for future payments option**, a payment_instrument object containing a token representing the card is created (AKA tokenized card) for this customer.

```
"payment_instrument": {
  "token": "6878cb7f-6515-47bf-bdd9-1408d270fdce"
}
```

At any time, you can fetch the list of tokenized cards of a customer by requesting them via [the list payment instruments] endpoint.

- **cURL**
- Node.js
- .NET
- Java
- Go
- Python
- Rust
- PHP

```
curl -X GET \
 "https://api.sumup.com/v0.1/customers/${CUSTOMER_ID}/payment-instruments" \
 -H "Authorization: Bearer $SUMUP_API_KEY" \
 -H "Content-Type: application/json;charset=UTF-8"
```

Processing Request Directly

If you want to provide a fully customized experience, you can process the request using [SumUp API directly]. In this case, you need to make sure to handle all legal requirements yourself. Critically, SumUp API expects the mandate object in the request. By providing the mandate object, **you are stating that you have obtained the consent of the customer**.
The mandate object establishes the authorization framework for storing and reusing card details. It defines what the card token will be used for (only recurring payments for the time being), while capturing the technical details of the tokenization request for security and compliance purposes.

```
{
 "payment_type": "card",
 "installments": 1,
 "card": {
  "name": "FIRSTNAME LASTNAME",
  "number": "4111111111111111",
  "expiry_year": "2023",
  "expiry_month": "01",
  "cvv": "123",
  "zip_code": "12345"
 },
 "mandate": {
  "type": "recurrent",
  "user_agent": "Chrome: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/77.0.3865.90 Safari/537.36.",
  "user_ip": "2001:0db8:85a3:0000:0a00:8a2e:0370:7300"
 }
}
```

Again, a payment_instrument object containing a token representing the card is created (AKA tokenized card) for this customer.

```
"payment_instrument": {
  "token": "6878cb7f-6515-47bf-bdd9-1408d270fdce"
}
```

At any time, you can fetch the list of tokenized cards of a customer by requesting them via the list payment instruments endpoint.

```
curl -X GET \
 "https://api.sumup.com/v0.1/customers/${CUSTOMER_ID}/payment-instruments" \
 -H "Authorization: Bearer $SUMUP_API_KEY" \
 -H "Content-Type: application/json;charset=UTF-8"
```

Retrieving Tokenized Card

Having successfully processed the checkout, a token representing the payment instrument (card) is created. You can now retrieve the checkout to find this token within a payment_instrument object for later recurrent payment.

Example response:

```
{
 "id": "cd36780e-f43d-4f22-1i9e-e32a1a1bafc8",
 "checkout_reference": "0BYNWLYC7KV",
 "amount": 3.51,
 "currency": "EUR",
 ...
 "payment_instrument": {
  "token": "2fa27578-e765-5dbh-aa97-d45d3d6cdfbb"
 }
}
```

Processing Recurring Payments

Having tokenized the customer's card, you can now process recurring payments by referencing the saved token and the associated customer. Both token and customer_id fields are required.

1. Create a checkout again. This time, it's for the actual payment. The previous checkout was for tokenizing the card only.
2. Process the checkout. Make sure to pass the following data (installments is only valid for the Brazilian market):

```
{
 "payment_type": "card",
 "installments": 1,
 "token": "{{CARD_TOKEN}}"
 "customer_id": "{{CUSTOMER_ID}}",
}
```

What's Next?

You may be interested in the following resources related to Online Payments:

- Handling Refunds
- Response Handling

Refunds

Walks through looking up transactions and issuing full or partial refunds.

Overview

In this guide, you will learn how to refund a transaction. You will go through the following steps:

1. Look up a transaction ID (Optional)
2. Refund a transaction by using one of the available options:
    - Option A: Make a full refund
    - Option B: Make a partial refund

When you complete these steps, the payment you have previously processed through SumUp will be refunded either partially or in full.

Before You Begin

Here are the things that you need in order to complete the steps in this guide:

- SumUp merchant account(Opens in a new tab) with completed account details(Opens in a new tab).
    - You can also use a sandbox merchant account.
- Registered client application with SumUp.
- Valid access token obtained with the Authorization code flow.
- You have processed a checkout and you have the checkout ID.

Note
Transactions are associated with active merchant user accounts. As a result, you *cannot* use an access token obtained via the Client credentials flow to complete the steps in this guide.

Steps

1. Look up a Transaction ID

Note
If you already have the ID of the transaction you want to refund, you can skip this step and continue with Step 2.

1. Make a GET request to the `https://api.sumup.com/v0.1/checkouts/{id}` endpoint, where the value of the `{id}` path parameter is the identifier of the checkout resource.

Example request:

- **cURL**
- Node.js
- .NET
- Java
- Go
- Python

- Rust
- PHP

```
curl -X GET \
  https://api.sumup.com/v0.1/checkouts/4ebc2ed7-bb8c-4d4d-a110-08fd31301bf2 \
  -H "Authorization: Bearer $SUMUP_API_KEY"
```

The response contains a JSON body with the full details of the processed checkout resource. You can find the transaction ID in the `id` attribute of the respective transaction resource (`664200af-2b62-4142-9c73-a2a505310d78` in the sample response below).

```
{
  "checkout_reference": "CO287866",
  ...
  "id": "4ebc2ed7-bb8c-4d4d-a110-08fd31301bf2",
  ...
  "transactions": [
    {
      "id": "664200af-2b62-4142-9c73-a2a505310d78",
      ...
    }
  ]
}
```

2. Refund a Transaction

- Option A: Make a full refund
- Option B: Make a partial refund

Make a Full Refund

1. Make a POST request with an empty request body to the `https://api.sumup.com/v0.1/me/refund/{txn_id}` endpoint, where the value of the `{txn_id}`path parameter is the identifier of the transaction resource.

Example request for the transaction with identifier `664200af-2b62-4142-9c73-a2a505310d78`:

- **cURL**
- Node.js
- .NET
- Java
- Go
- Python
- Rust
- PHP

```
curl -X POST \
  https://api.sumup.com/v0.1/me/refund/19aa3cca-89f6-42d2-b462-463b0b53e959 \
  -H "Authorization: Bearer $SUMUP_API_KEY"
```

The response returns a 204 HTTP status code and contains no body.

Make a Partial Refund

1. Make a POST request to the `https://api.sumup.com/v0.1/me/refund/{txn_id}` endpoint, where the value of the `{txn_id}` path parameter is the identifier of the transaction resource.

Unlike the option for a full refund, the request body for partial refunds should be a JSON object with the amount you want to refund for the transaction.

Example request for a partial refund for the amount of 24.42 EUR:

- **cURL**
- Node.js
- .NET
- Java
- Go
- Python
- Rust
- PHP

```
curl -X POST \
  https://api.sumup.com/v0.1/me/refund/19aa3cca-89f6-42d2-b462-463b0b53e959 \
  -H "Authorization: Bearer $SUMUP_API_KEY" \
  -H 'Content-Type: application/json' \
  -d '{"amount": 24.42}'
```

The response returns a 204 HTTP status code and contains no body.

Result

You have successfully refunded a transaction (either partially or in full) for a payment you previously processed. The refunded amount will be credited to the same payment method the customer had used to pay with in the original transaction. The processing fees associated with the original transaction are not returned.

Response Handling

Explains how to interpret SumUp API responses.

Overview

Once an API call is submitted to SumUp, you will receive a standard HTTP response informing you if your request has been successful or not. In most cases the HTTP codes are accompanied by a response body in JSON(Opens in a new tab) format.

Codes in the `2xx` range indicate success,`4xx` range indicates errors where the information provided results in a failure. Codes in the `5xx` range are rare and indicate server-side errors.

Successful Requests

Successfully processed requests return one of the following HTTP status codes: `200 OK`, `201 Created`, `202 Accepted`, `204 No Content`. Depending on the use case response bodies may not be present at all. A response body can look like this:

{
 "next_step": {
  "url": "https://dummy-3ds-gateway.com/cap?RID=1233&VAA=A",
  "method": "POST",
  "redirect_url": "https://mysite.com/completed_purchase",
  "mechanism": "iframe",
  "payload": {
   "PaReq":
"eJxVUttu2zAM/RXDr4MjyY5dO6BVuE27FZuDZHGG9VGRmMSFb/..f16+jLt/gPhUvGGw==",
   "MD": "b1a536c0-29b9-11eb-adc1-0242ac120002"
  }
 }
}

Client-Side Issues

4xx are occasions where you can take action to correct your application requests. Typically, these codes are returned upon user error, such as invalid API calls, incorrect values, missing parameters, etc. The response you receive will provide an indication to the root cause that triggered it. Here's an example of a 4xx response:

{
 "message": "Validation error",
 "error_code": "MISSING",
 "param": "merchant_code"
}

Server-Side Issues

5xx stand for server errors. Although rare, if you receive such code, we recommend retrying your request. Should the returned codes continue to be in the 5xx range, reach out to us.